



Universidad autónoma de baja California

Ingeniería en computación

Sistemas embebidos

Practica N.2 sincronización de tarea (juego)

Evangelina Lara Camacho

Garcia Chávez Erik 01275863

Viernes 26 de septiembre del 2025

Función **xTaskCreatePinnedToCore()**

es una función de FreeRTOS para ESP32 que te permite crear tareas y asignarlas específicamente a un núcleo del procesador. Evitando sobrecargar un solo núcleo, **Mejora el rendimiento** al distribuir el trabajo

```
xTaskCreatePinnedToCore(  
    funcion_tarea,    // La función que contiene el código de la tarea  
    "NombreTarea",    // Nombre para identificar la tarea  
    2048,             // Memoria para la tarea (en palabras)  
    NULL,             // Parámetros que le pasas a la tarea  
    1,               // Prioridad (1=baja, números mayores=alta)  
    NULL,             // Variable para manejar la tarea después  
    0                 // Núcleo: 0=Core 0, 1=Core 1  
);
```

Función **xEventGroupSync()**

xEventGroupSync es una función sincronización que hace dos cosas al mismo tiempo:

1. Pone bits (banderas) en un grupo de eventos
2. Espera a que otros bits específicos se activen

Ejemplo:

```
EventBits_t xEventGroupSync(  
    EventGroupHandle_t xEventGroup, // Grupo de eventos a usar  
    const EventBits_t uxBitsToSet,  // Bits que VOY a activar  
    const EventBits_t uxBitsToWaitFor, // Bits que ESPERO que estén activos  
    TickType_t xTicksToWait         // Tiempo máximo a esperar  
);
```

Desarrollo de la actividad:

Material a necesitar:

1 esp32

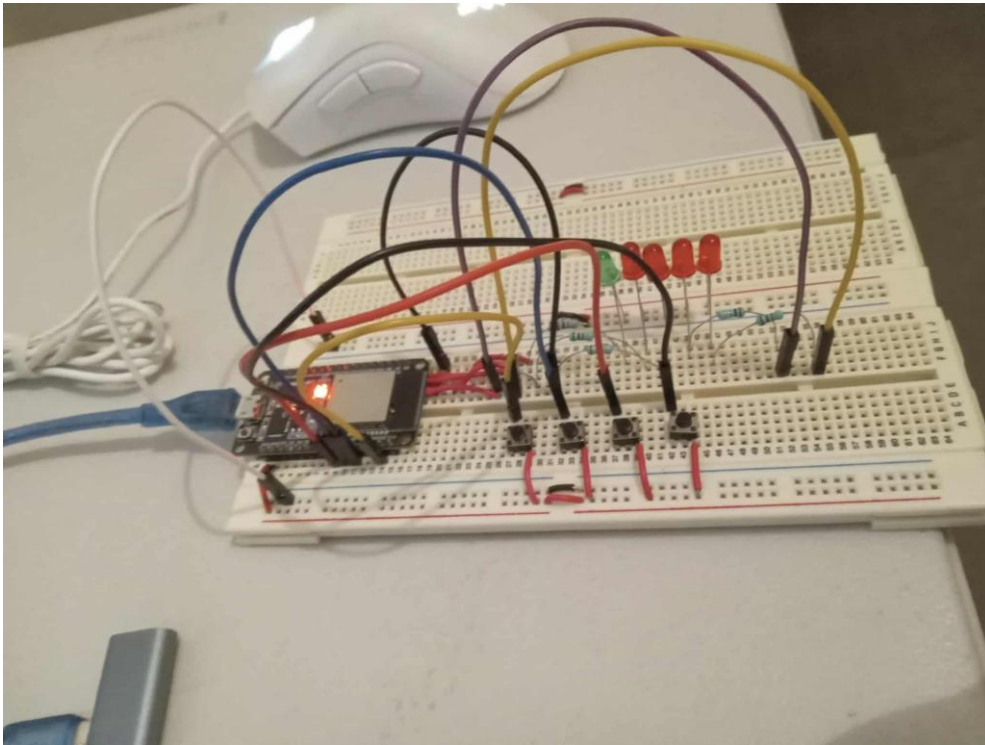
4 push bottoms

5 leds

5 resistencias de 1Kohm

Jumpers – cable

Circuito armado:



El juego cuando inicia muestra primero un texto antes de iniciar en donde se pide que presione el botón de inicio primero antes de iniciar el juego.

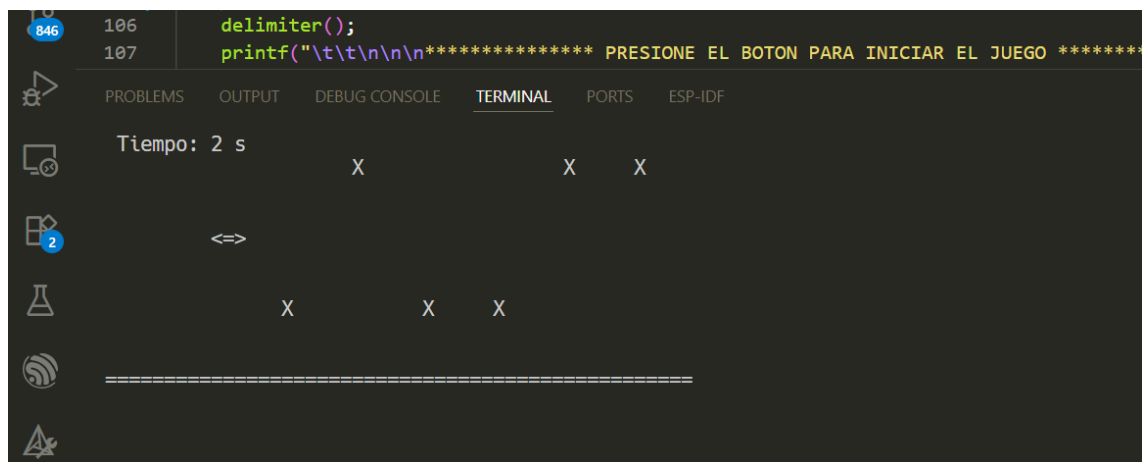
Esto está configurado con un grupo de evento ya que solo quiero indicar el estado de una condición.

```
I (226) heap_init: Initializing. RAM available for dynamic allocation:
I (232) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (237) heap_init: At 3FFB32A0 len 0002CD60 (179 KiB): DRAM
I (242) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM
I (248) heap_init: At 3FFE4350 len 00018CB0 (111 KiB): D/IRAM
I (253) heap_init: At 40080F64 len 0001209C (72 KiB): IRAM
I (260) spi_flash: detected chip: generic
I (262) spi_flash: flash to: dio
W (265) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (278) main_task: Started on CPU0
I (288) main_task: Calling app_main()
configUSE_16_BITS_TICK 0
=====

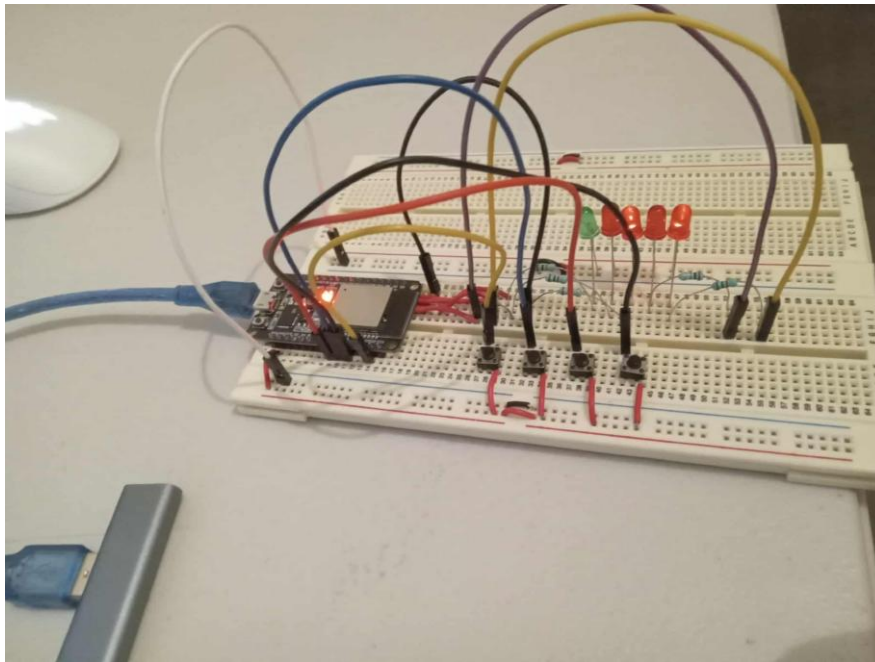
***** PRESIONE EL BOTON PARA INICIAR EL JUEGO *****

=====
I (298) main_task: Returned from app_main()
```

Una vez que se presiona el botón que inicia el juego, el juego iniciara



En el extremo superior izquierdo nos muestra el tiempo transcurrido en segundos. Al igual que nos lo está mostrando de manera binario por los GPIO



En vehículo representado por los caracteres ascii (<=>) se puede mover en 3 carriles, arriba, en medio y abajo

```

70  void app_main(void)
104      //minitarea que se encarga de iniciar el juego
105      //xTaskCreate(task_init_game,"task_init_game",2048,NULL,5,NULL);
106      delimiter();
107      printf("\t\t\t\n\n***** PRESIONE EL BOTON PARA INICIAR EL JUEGO

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF

Tiempo: 15 s

```

          X      <=>      X
      X              X
          X              X
=====

```

Cuando este inicia, inicia en el carril de en medio y los obstáculos aparecen de manera aleatoria entre estos 3 carriles, dejando un hueco de 5 espacios para que el vehículo pueda pasar entre los obstáculos en los carriles.

La forma de terminar con el juego es perdiendo, en dado caso muestra un mensaje que “suerte a la próxima”

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

Tiempo: 4 s
    <=>      X          X      X

                X          X

=====
=====

=== ¡suerte a la proxima===

***** PRESIONE EL BOTON PARA INICIAR DE NUEVO *****

=====
```

O bien con el botón de salida del juego que indica “hasta luego”

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

Tiempo: 2 s

                X      X

    <=>      X

        X      X          X

=====
=====

=== HASTA LUEGO ===

***** PRESIONE EL BOTON PARA INICIAR DE NUEVO *****

=====
```

Explicación de código:

Tenemos 2 definiciones en nuestro archivo .h que indican en que posición va a spawnear nuestro vehículo y cunado es el espacio mínimo que queremos dejar entre obstáculos así que podemos cambiar estas definiciones para cambiar la posición y hasta la dificultad dejando un margen más corto de error.

```
1 #define SPAWN_OBJ 5 //el spawn de nuestro vehiculo
2 #define SPACE_OBJ 6 //el espacio minimo que debe de haber entre objetos
3
```

Tenemos otra definición para el formato de la “pantalla” en este caso es de 50 x 10, por lo que esto podría cambiar si así lo quisiéramos

```
1 #define WIDTH_SCRE 50
2 #define HEIGHT_SCRE 10
```

Para hacer más sencillo el establecer 3 carriles, agregue las siguientes definiciones, estos valores solo tienen sentido si es una matriz de 10 espacios de alto, estas definen que índice de la matriz representan el carril de arriba, en medio y abajo.

```
1 #define LANE_TOP 1 // Carril superior
2 #define LANE_CENTER 4 // Carril central
3 #define LANE_BOTTOM 7 // Carril inferior
```

De ahí en más solo tenemos definiciones de nuestras funciones.

Archivo main.c

Establecemos 2 matrices, la matriz **obstacle** es donde guardaremos en que posición de los 3 carriles posibles habrá un obstáculo este se genera de manera random usando la función **rand()** y la matriz **canvas** es la que presenta por así decirlo nuestra “**pantalla de juego**” es en donde se estará mostrando los obstáculos y nuestro vehículo, así como el tiempo que ha pasado en segundos

```
1 volatile int obstacle[3][WIDTH_SCRE];
2 volatile char canvas[HEIGHT_SCRE][WIDTH_SCRE];
```

En nuestra función que está tratando la interrupción de cunado se presiona un botón, para los botones, **_UP_**, **_DOWN_**, **_FINISH_**, que son los que indican si se mueve hacia arriba, hacia abajo, o finalizo el juego, pero antes es necesario un proceso antirebote por software, lo que es diferente en el caso de **_INIT_** porque?, mi idea es que se quedase en la “pantalla de inicio” hasta que se presionase iniciar el juego, entonces para esto use un grupo de eventos que indican en la tarea principal cunado se presionó el botón entonces podrá continuar con el juego. Esto solo apsara en cuanto se inicia el juego, en los 4 casos enviamos que botón se presionó a través de colas.

```
1
2 void IRAM_ATTR gpio_isr_handler(void *args){
3     uint32_t pin_number = (uint32_t)args;
4     uint64_t current_time = esp_timer_get_time();
5
6     switch(pin_number) {
7         case _UP_:{
8             if((current_time - LAST_PRESS_UP_) > DEBOUNCE_TIME){
9
10                 if(gpio_get_level(pin_number) == 0){
11                     LAST_PRESS_UP_ = current_time;
12                     xQueueSendFromISR(handlerQueue, &pin_number, NULL);
13                 }
14             }
15             }break;
16
17         case _DOWN_:{
18             if((current_time - LAST_PRESS_DOWN_) > DEBOUNCE_TIME){
19                 if(gpio_get_level(pin_number) == 0){
20                     LAST_PRESS_DOWN_ = current_time;
21                     xQueueSendFromISR(handlerQueue, &pin_number, NULL);
22                 }
23             }
24             }break;
25
26         case _FINISH_:{
27             if((current_time - LAST_PRESS_FINISH_) > DEBOUNCE_TIME){
28                 if(gpio_get_level(pin_number) == 0){
29                     LAST_PRESS_FINISH_ = current_time;
30                     xQueueSendFromISR(handlerQueue, &pin_number, NULL);
31                 }
32             }
33             }break;
34
35         case _INIT_:
36             if ((current_time - LAST_PRESS_INIT_) > DEBOUNCE_TIME) {
37                 if(gpio_get_level(pin_number) == 0){
38                     LAST_PRESS_INIT_ = current_time;
39                     if (inicio == false) {
40                         inicio = true;
41                         BaseType_t xHigherPriorityTaskWoken = pdFALSE;
42                         xEventGroupSetBitsFromISR(event_group, ready, &xHigherPriorityTaskWoken);
43                         xQueueSendFromISR(handlerQueue, &pin_number, NULL);
44                         if (xHigherPriorityTaskWoken) {
45                             portYIELD_FROM_ISR();
46                         }
47                     }
48                 }
49             }break;
50     }
51 }
52
```


Tarea principal:

Como se indicó anteriormente lo primero que verifica la tarea es que el bit que indica que se activó el inicio se haya activado, para eso se hace uso de la función **xEventGroupWaitBits()** en donde si no se ha activado aun la tarea esta inactiva, una vez el bit se establezca la tarea puede seguir.

Lo primero que se hace en entorno al juego es generar los obstáculos estos se generan de manera aleatoria.

Una vez generados en la matriz **obstacle** ya se tiene mapeado en que parte de la matriz **canvas** estarán los obstáculos, estos obstáculos se generan de nuevo para cada “**cambio de pantalla**”

Lo siguiente es que está esperando a que botón es el que se presionara después esto se recibe de la ISR a través de la cola. Esto hará ir al vehículo hacia arriba o hacia abajo limitando el rango en que se puede desplazar el vehículo, porque si no se limita puede irse más allá de la matriz o de la “pantalla” permitida y causara un bug. Por lo que indicamos condiciones por el caso que el jugador este presionando constantemente un botón ya sea el de subir o el de bajar.

Si no se decidió terminar con el juego lo que sigue el mover el vehículo, esto lo hacemos con la función **move_vehicle()** que mueve el vehículo dentro del **canvas** en la dirección del eje x. por lo que los obstáculos se quedan estáticos es el vehículo el que se desplaza a través de la pantalla, se me hizo mucho más sencillo mover 1 solo objeto a mover varios.

Después de eso checamos si en ese movimiento la posición en la que esta ahora el vehículo ya se cualquiera de las 3 partes que lo conforman no han “**chocado**” con algún obstáculo, la presentación en el vector de **obstacle** lo tengo representado como 1, en toda la matriz en donde aparezca 1 indica que es un obstáculo. Si es así termina el juego si no continua

Por lo que presenta en pantalla el juego actual, el cual se está constantemente refrescando con un delay que podemos ajustar si lo queremos más rápido o más lento

```

1 void task_game_main(void *params){
2
3     while(1){
4
5
6         //aquí vamos a esperar a que el evento en que recibe la senial de active
7         xEventGroupWaitBits(event_group, ready, false, true, portMAX_DELAY);
8
9         int step;
10        //con esto estamos idnicnado que vamos a querer que la tarea se ejecuter siempre y cunado no se pierda
11        bool game_over=false;
12
13        // Limpiar completamente la terminal
14        printf("\033[2J\033[H\033[3J"); // Limpia pantalla y buffer
15        printf("\033[?251");
16
17        vehicle_x = SPAWN_OBJ;
18
19        generate_obstacle();
20
21
22        while(!game_over){
23
24            if(xQueueReceive(moveObjectQueue, &step, 50)){
25                switch(step){
26                    case _UP_:{
27                        if(current_line > 0){
28                            current_line--;
29
30                        }
31                    }break;
32
33                    case _DOWN_:{
34                        if(current_line < 2){
35                            current_line++;
36
37                        }
38                    }break;
39
40                    case _FINISH_:{
41                        printf("\n\n=== HASTA LUEGO ===\n");
42                        game_over =true;
43                        break;
44                    }
45                }
46
47            }
48            move_vehicle();
49
50            if(check_collision()){
51                printf("\n\n=== ¡suerte a la proxima===\n");
52                game_over = true;
53            }
54
55            update_lanes();
56
57            display_games();
58            vTaskDelay(game_speed/portTICK_PERIOD_MS);
59
60        }
61        reset_game_state();
62    }
63 }
64

```

Funcion **generate_obstacle()**:

Esta función de lo que se encarga es de establecer en que posición estará cada obstáculo, de manea random elegimos cualquiera de los 3 carriles que tenemos, que antes necesitamos establecer una regla, esta regla está definida en la función **check_space_available()**. En la cual establece la distancia mínima que deben de estar los obstáculos esto para que no genere tantos tan juntos e impidan que se pueda pasar, si se cumple la función manda 1 indicando que es viable insertar el obstáculo en la dirección prevista, para esto se ira recorriendo el ancho de la pantalla para abarcar la mayoría de los lugares.

```
1 void generate_obstacle(void){
2
3     memset((void*)canvas, ' ', sizeof(canvas));
4     memset((void*)obstacle, 0, sizeof(obstacle));
5
6
7     for(int x = WIDTH_SCRE - 35; x<WIDTH_SCRE; x+=6){
8
9         if(x < WIDTH_SCRE){
10
11             int lane = rand()%3;
12             if(check_space_avilable(lane,x) == 1){
13                 obstacle[lane][x] = 1;
14             }
15         }
16     }
17 }
```

Funcion **check_space_avilabe()**:

La función recibe la fila y la columna en donde se planea insertar el obstáculo, pero debe de haber un espacio por el cual el vehículo pueda pasar, el espacio está definido por la macro **SPACE_OBJ** en mi caso es de 5 espacios, por lo que le estoy diciendo, en el caso que se quiera insertar en el carril superior revisa en la fila inferior si en los siguientes **SPACE_OBJ** no hay otro obstáculo, esto dentro de la matriz **obstacle** si es así regresa 1 indicando que se puede insertar si no, regresa 0 indicando que no.

En el caso del carril de en medio es parecido solo que aquí tengo que checar al menos 1 carril debe de cumplir con la condición, revisa arriba y abajo si en los siguientes **SPACE_OBJ** espacios no hay obstáculo.

En el caso del carril inferior es la misma técnica solo que revisa el carril de en medio.

```

1  int check_space_availabe(int row,int col){
2
3
4      switch(row){
5
6          case 0:{
7
8              for(int x = col; x < col + SPACE_OBJ && x < WIDTH_SCRE;x++){
9
10                 if(obstacle[1][x] == 1){
11                     return 0;
12                 }
13             }
14             return 1;
15         }break;
16
17         case 1:{
18
19             bool top_free= true;
20             bool bottom_free = true;
21
22             for(int x = col; x < col + SPACE_OBJ && x < WIDTH_SCRE;x++){
23                 if(obstacle[2][x] == 1){
24                     bottom_free = false;
25                 }
26                 if(obstacle[0][x]== 1){
27                     top_free = false;
28                 }
29             }
30
31             return (bottom_free || top_free)? 1 : 0;
32         }break;
33
34         case 2:{
35
36             for(int x = col; x < col + SPACE_OBJ && x < WIDTH_SCRE;x++){
37
38                 if(obstacle[1][x] ==1 ){
39                     return 0;
40                 }
41             }
42             return 1;
43         }break;
44     }
45     return 0;
46 }
47
48

```

Función **move_vehicle()**:

Esta función lo único que hace es ir avanzando en la posición que está el vehículo para mostrarlo en pantalla en el caso se llegue al final genera una nueva matriz de obstáculos.

```
1
2 void move_vehicle(void){
3     vehicle_x++;
4
5
6     if (vehicle_x >= WIDTH_SCRE - 3) {
7         vehicle_x = SPAWN_OBJ;
8         generate_obstacule();
9     }
10 }
```

Función **check_collision()**

Mapeamos cada parte del vehículo para saber si algunas de las 3 partes que lo conforman tocan algún obstáculo. Como la variable global de `current_line` indica en que carril se está dirigiendo, esta variable solo puede ir de 0 – 2, por lo que le pasamos el carril actual y en donde se encuentra el vehículo en el eje x, si alguna de las 3 piezas está en la misma posición en donde hay un 1 en la matriz indica que colisiono si no, no lo hizo y manda 0 indicando que el juego aun continua.

```
1 int check_collision(void) {
2
3     for (int i = 0; i < 3; i++) {
4         int check_x = vehicle_x + i;
5
6
7         if (check_x >= 0 && check_x < WIDTH_SCRE) {
8
9             if (obstacle[current_line][check_x] == 1) {
10                 return 1;
11             }
12         }
13     }
14
15     return 0;
16 }
17
```

Funcion **update_lanes()**:

En esta funcion actualizamos lo que debería de mostrar el matiz **canvas**, lo primero es que “**anclamos**” en donde se mostrara el tiempo en la pantalla.

De pues lo que hacemos es mapear en donde se ubican nuestros obstáculos y los presentamos en la “**pantalla**”

Al igual que dibujamos al vehículo dentro de la matriz de **canvas**

```
1 void update_lanes(){
2
3
4     memset((void*)canvas, ' ', sizeof(canvas));
5
6     char time_buffer[25];
7     sprintf(time_buffer, "Tiempo: %lu s", elapsed_seconds);
8     memcpy(&canvas[0][1], time_buffer, strlen(time_buffer));
9
10    for(int x =0; x< WIDTH_SCRE; x++){
11
12        if(obstacle[0][x]) canvas[LANE_TOP][x]= 'X';
13        if(obstacle[1][x]) canvas[LANE_CENTER][x]= 'X';
14        if(obstacle[2][x]) canvas[LANE_BOTTOM][x]= 'X';
15    }
16
17
18    int vehicle_lane;
19    switch(current_line) {
20        case 0: vehicle_lane = LANE_TOP; break;
21        case 1: vehicle_lane = LANE_CENTER; break;
22        case 2: vehicle_lane = LANE_BOTTOM; break;
23        default: vehicle_lane = LANE_CENTER;
24    }
25
26    if (vehicle_x >= 0 && vehicle_x + 2 < WIDTH_SCRE) {
27        canvas[vehicle_lane][vehicle_x] = '<';
28        canvas[vehicle_lane][vehicle_x + 1] = '=';
29        canvas[vehicle_lane][vehicle_x + 2] = '>';
30    }
31
32 }
33
```

Función **display_games()**:

Esta es la función que el realizad imprime en pantalla la matriz y todo el contenidos del juego.



```
1  void display_games(void){
2
3
4
5      static bool first_display = true;
6
7      if(first_display) {
8          printf("\033[2J\033[H");
9          first_display = false;
10     } else {
11         printf("\033[H");
12
13         for(int y= 0; y < HEIGHT_SCRE; y++){
14
15             for(int x = 0; x< WIDTH_SCRE; x++){
16                 putchar(canvas[y][x]);
17             }
18             putchar('\n');
19         }
20         delimiter();
21
22         fflush(stdout);
23
24
25     }
26
```

Dificultades:

Con lo que tuve un poco de problemas fue el entender a los grupo de eventos, las colas ya había trabajado con ellas y ya tenía una noción, pero en esta practica me quedaron mas claras, pero lo que no entendía muy bien era al grupo de eventos, me consto un poco lograr entender lo que representaba cada parámetro, así como el manejar las matices, tenía tiempo que no programaba matrices siempre fue algo en donde batalle un poco, pero con la practica pude entender varias cosas que antes no comprendía, creo que al final de todo solo es cuestión de practicar más.

